

REPORT ON THE CRYPTO BOT

Source code:

```
from binance.client import Client
import logging
class BinanceFuturesBot:
    def __init__(self, api_key, api_secret, testnet=True):
        self.client = Client(api_key, api_secret, testnet=testnet)
        self.client.FUTURES_URL = 'https://testnet.binancefuture.com/fapi' if testnet else self.client.FUTURES_URL
        logging.basicConfig(filename='bot.log', level=logging.INFO,
                            format='%(asctime)s : %(levelname)s : %(message)s')
    def place_market_order(self, symbol, side, quantity):
        try:
            order = self.client.futures_create_order(
                symbol=symbol,
                side=side,
                type='MARKET',
                quantity=quantity
            )
            logging.info(f"Market order placed: {order}")
            print(f"Market order placed: {order}")
            return order
        except Exception as e:
            logging.error(f"Market order failed: {e}")
            print(f"Market order failed: {e}")
            return None
    def place_limit_order(self, symbol, side, quantity, price):
        try:
            order = self.client.futures_create_order(
                symbol=symbol,
```

The Binance Futures Order Bot project is a command-line interface (CLI) trading bot designed to automate the placement of various types of orders on the Binance USDT-M Futures Testnet. The bot supports essential order types such as market orders and limit orders, as well as advanced strategies like One-Cancels-the-Other (OCO) and Time-Weighted Average Price (TWAP) orders.

The project is structured modularly, with different scripts for each order type, promoting clarity and ease of maintenance. It uses the official Binance API client configured to interact with the Binance Futures Testnet environment, ensuring safe and risk-free experimentation. Input validation and exception handling are integral parts of the bot, allowing users to input trading symbols, order sides, quantities, prices, and stop prices interactively. The bot provides immediate console feedback on order placements and errors, while also logging detailed transaction records in a structured log file for auditing and debugging purposes.

This approach enables traders and developers to automate common trading tasks, test different strategies, and gain practical experience with Binance Futures API workflows. The bot serves as a foundational platform that can be extended with more complex order types, better risk management, and integration of market data analytics, making it a versatile tool for algorithmic trading development on Binance Futures.

```

        side=side,
        type='LIMIT',
        quantity=quantity,
        price=price,
        timeInForce='GTC'
    )
    logging.info(f"Limit order placed: {order}")
    print(f"Limit order placed: {order}")
    return order
except Exception as e:
    logging.error(f"Limit order failed: {e}")
    print(f"Limit order failed: {e}")
    return None

def place_stop_market_order(self, symbol, side, quantity, stop_price):
    try:
        order = self.client.futures_create_order(
            symbol=symbol,
            side=side,
            type='STOP_MARKET',
            quantity=quantity,
            stopPrice=stop_price,
            timeInForce='GTC'
        )
        logging.info(f"Stop market order placed: {order}")
        print(f"Stop market order placed: {order}")

```

```

    return order
except Exception as e:
    logging.error(f"Stop market order failed: {e}")
    print(f"Stop market order failed: {e}")
    return None

def main():
    print("Welcome to Binance Futures Bot")
    print("Order types: MARKET, LIMIT, STOP_MARKET")
    order_type = input("Enter order type: ").strip().upper()
    symbol = input("Enter trading symbol (e.g., BTCUSDT): ").strip().upper()
    side = input("Enter side (BUY or SELL): ").strip().upper()

    quantity_input = input("Enter quantity: ").strip()
    try:
        quantity = float(quantity_input)
    except ValueError:
        print("Invalid quantity. Must be a number.")
        return

    API_KEY = "6fb8c22049dcdf1906093a75f421029449969dcb85391df8e9271d8fe1e3b227"
    API_SECRET = "00cde179d8253511429a0207defc11712be7fa729c124c8999776c6c93eabcfb"

    bot = BinanceFuturesBot(API_KEY, API_SECRET, testnet=True)

    if order_type == "MARKET":

```

```

bot = BinanceFuturesBot(API_KEY, API_SECRET, testnet=True)

if order_type == "MARKET":
    bot.place_market_order(symbol, side, quantity)

elif order_type == "LIMIT":
    price_input = input("Enter limit price: ").strip()
    try:
        price = float(price_input)
    except ValueError:
        print("Invalid price. Must be a number.")
        return
    bot.place_limit_order(symbol, side, quantity, price)

elif order_type == "STOP_MARKET":
    stop_price_input = input("Enter stop price: ").strip()
    try:
        stop_price = float(stop_price_input)
    except ValueError:
        print("Invalid stop price. Must be a number.")
        return
    bot.place_stop_market_order(symbol, side, quantity, stop_price)
else:
    print(f"Unsupported order type: {order_type}. Choose MARKET, LIMIT or STOP_MARKET.")

if __name__ == "__main__":
    main()

```

OUTPUT:

```

Welcome to Binance Futures Bot
Order types: MARKET, LIMIT, STOP_MARKET
Enter order type: market
Enter trading symbol (e.g., BTCUSDT): ethusdt
Enter side (BUY or SELL): buy
Enter quantity: 5.99
Market order placed: {'orderId': 5006536626, 'symbol': 'ETHUSDT', 'status': 'NEW', 'clientOrderId': 'x-Cb7ytekJ925fba889e20cea4979b96', 'pr
ice': '0.00', 'avgPrice': '0.00', 'origQty': '5.990', 'executedQty': '0.000', 'cumQty': '0.000', 'cumQuote': '0.00000', 'timeInForce': 'GTC
', 'type': 'MARKET', 'reduceOnly': False, 'closePosition': False, 'side': 'BUY', 'positionSide': 'BOTH', 'stopPrice': '0.00', 'workingType
': 'CONTRACT_PRICE', 'priceProtect': False, 'origType': 'MARKET', 'priceMatch': 'NONE', 'selfTradePreventionMode': 'EXPIRE_MAKER', 'goodTill
Date': 0, 'updateTime': 1757366582418}
PS C:\Users\HP\Desktop>

```

